```
# ================================================================================
# [PROJE ADIMI 1] ORTAM KURULUMU VE VERİ AKTARIMI
# ================================================================================
# Bu hücrenin amacı:
# 1. Derin öğrenme modelimiz (YOLO) için gerekli ultralytics kütüphanesini kurmak.
# 2. Google Drive'a bağlanarak oradaki NEU-DET.zip dosyamızı hızlı bellek olan Colab'e çıkarmak.
# ================================================================================

# YOLOv8 kütüphanesini kuruyoruz
!pip install ultralytics -q # -q parametresi gereksiz logları gizler, temiz çıktı verir

from google.colab import drive
import os

# Google Drive'ımızı Colab çalışma ortamına bağlıyoruz
print("Google Drive'a bağlanılıyor...")
drive.mount('/content/drive')

# Daha önce çıkarılmış veri varsa temizleyelim (çakışmayı önlemek için)
if os.path.exists('/content/dataset'):
    !rm -rf /content/dataset

# Drive'daki zip dosyasını Colab'in kendi diskine (dataset klasörüne) çıkarıyoruz.
# DİKKAT: "/content/drive/MyDrive/NEU-DET.zip" yolunu kendi zip dosyanızın adına göre güncelleyin.
print("Veri seti arşivden çıkarılıyor, lütfen bekleyin...")
!unzip -q "/content/drive/MyDrive/NEU-DET.zip" -d "/content/dataset"

print("✅ Kurulum ve veri aktarımı başarıyla tamamlandı!")
```

```
Google Drive'a bağlanılıyor...
Mounted at /content/drive
Veri seti arşivden çıkarılıyor, lütfen bekleyin...
✅ Kurulum ve veri aktarımı başarıyla tamamlandı!
```

```
# ================================================================================
# [PROJE ADIMI 2] KEŞİFÇİ VERİ ANALİZİ (EDA) VE VERİ GÖRSELLEŞTİRME
# ================================================================================
# Bu hücrenin amacı:
# 1. Veri setindeki (XML dosyaları) etiketleri analiz etmek.
# 2. Hangi kusur sınıfından kaç adet bulunduğunu saymak.
# 3. Proje raporumuzun "Veri Seti Analizi" bölümünde kullanmak üzere şık bir grafik oluşturmak.
# ================================================================================

import xml.etree.ElementTree as ET
import glob
import matplotlib.pyplot as plt
import seaborn as sns

# XML dosyalarının bulunduğu klasör yolu (Zip'ten çıkan yapıya göre güncelleyin)
# Eğer zip içinden direkt ANNOTATIONS çıkıyorsa burası doğrudur.
xml_dir = '/content/dataset/NEU-DET/ANNOTATIONS'
xml_files = glob.glob(f'{xml_dir}/*.xml')

# Kusur sınıflarını saymak için bir sözlük (dictionary) oluşturuyoruz
class_counts = {
    'crazing': 0,
    'inclusion': 0,
    'patches': 0,
    'pitted_surface': 0,
    'rolled-in_scale': 0,
    'scratches': 0
}

# Tüm XML dosyalarını tek tek okuyup içindeki hataları sayıyoruz
for xml_file in xml_files:
    try:
        tree = ET.parse(xml_file)
        root = tree.getroot()
        for obj in root.iter('object'):
            cls_name = obj.find('name').text
            if cls_name in class_counts:
                class_counts[cls_name] += 1
    except Exception as e:
        continue # Hatalı dosya varsa atla

# ================ RAPOR İÇİN GRAFİK ÇİZİMİ ================
plt.figure(figsize=(10, 6)) # Grafiğin boyutu
sns.set_theme(style="whitegrid") # Akademik ve şık bir arka plan teması

# Sütun grafiğini oluşturuyoruz
bars = sns.barplot(x=list(class_counts.keys()), y=list(class_counts.values()), palette="viridis")

# Sütunların üzerine rakamları yazdırıyoruz
for bar in bars.patches:
    bars.annotate(format(bar.get_height(), '.0f'),
                  (bar.get_x() + bar.get_width() / 2, bar.get_height()),
```

```
                ha='center', va='center', size=11, xytext=(0, 8),
                textcoords='offset points')

plt.title('NEU-DET Veri Seti Sınıf Dağılımı', fontsize=16, fontweight='bold')
plt.xlabel('Kusur Sınıfları (Defect Classes)', fontsize=12)
plt.ylabel('Etiket Sayısı (Instance Count)', fontsize=12)
plt.xticks(rotation=45) # İsimler birbirine girmesin diye 45 derece eğiyoruz
plt.tight_layout()

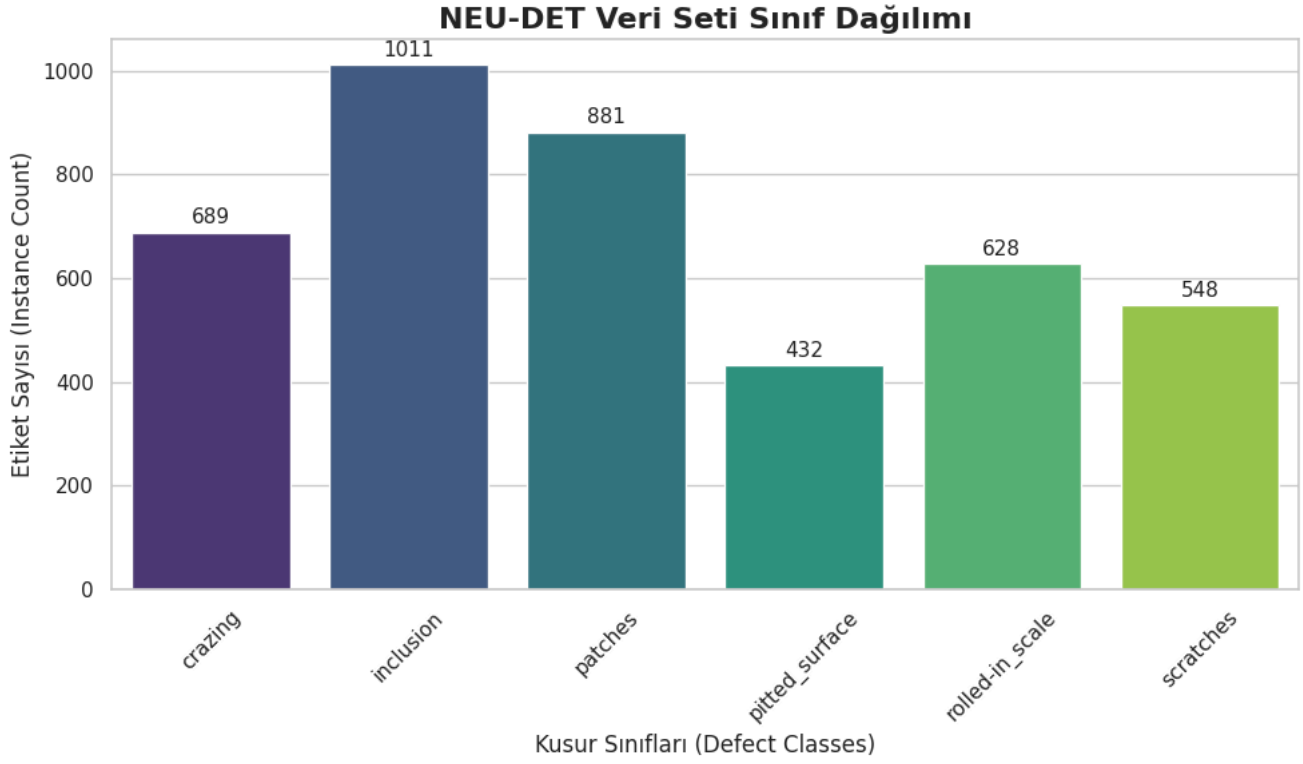
# Grafiği rapora koymak üzere resim olarak kaydediyoruz
plt.savefig('/content/sinif_dagilimi.png', dpi=300)
print("✅ Veri analizi tamamlandı! 'sinif_dagilimi.png' dosyası Colab'e kaydedildi. Sağ tıklayıp bilgisayarınıza indirebilirsiniz.")

# Grafiği Colab ekranında gösterelim
plt.show()
```

```
/tmp/ipykernel_2454/341003423.py:47: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=F

  bars = sns.barplot(x=list(class_counts.keys()), y=list(class_counts.values()), palette="viridis")
✅ Veri analizi tamamlandı! 'sinif_dagilimi.png' dosyası Colab'e kaydedildi. Sağ tıklayıp bilgisayarınıza indirebilirsiniz.
```




```
# ==============================================================================
# [PROJE ADIMI 3] KUSUR BOYUT ANALİZİ (BOUNDING BOX SCATTER PLOT)
# ==============================================================================
# Bu hücrenin amacı:
# 1. Her bir kusurun (çatlak, çizik, leke) resim üzerinde ne kadar alan kapladığını hesaplamak.
# 2. Hangi kusurların model için "Küçük Hedef (Small Target)" olduğunu kanıtlayan bir grafik (Scatter Plot) çizmek.
# Bu grafik, modelimizin neden ilk başta 'crazing' sınıfında zorlandığını bilimsel olarak açıklayacaktır.
# ==============================================================================

import xml.etree.ElementTree as ET
import glob
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

xml_dir = '/content/dataset/NEU-DET/ANNOTATIONS'
xml_files = glob.glob(f'{xml_dir}/*.xml')

# Verileri tutacağımız liste
bbox_data = []

# Tüm XML dosyalarındaki kutuların genişlik ve yüksekliğini alıyoruz
for xml_file in xml_files:
    try:
        tree = ET.parse(xml_file)
        root = tree.getroot()
        for obj in root.iter('object'):
            cls_name = obj.find('name').text
            xmlbox = obj.find('bndbox')

            # Kutunun genişlik (w) ve yüksekliğini (h) pikseller üzerinden hesapla
            w = float(xmlbox.find('xmax').text) - float(xmlbox.find('xmin').text)
            h = float(xmlbox.find('ymax').text) - float(xmlbox.find('ymin').text)

            bbox_data.append({'Sinif': cls_name, 'Genislik': w, 'Yukseklik': h})
```

```
    except Exception:
        continue

# Veriyi bir Pandas DataFrame'ine çeviriyoruz (Grafik çizmek için en iyisi)
df_bbox = pd.DataFrame(bbox_data)

# ================ RAPOR İÇİN GRAFİK ÇİZİMİ ================
plt.figure(figsize=(12, 8))
sns.set_theme(style="darkgrid")

# Her sınıf için farklı renkte noktalar oluşturuyoruz (Scatter Plot)
scatter = sns.scatterplot(data=df_bbox, x="Genislik", y="Yukseklik", hue="Sinif",
                          alpha=0.6, palette="bright", s=50, edgecolor="k")

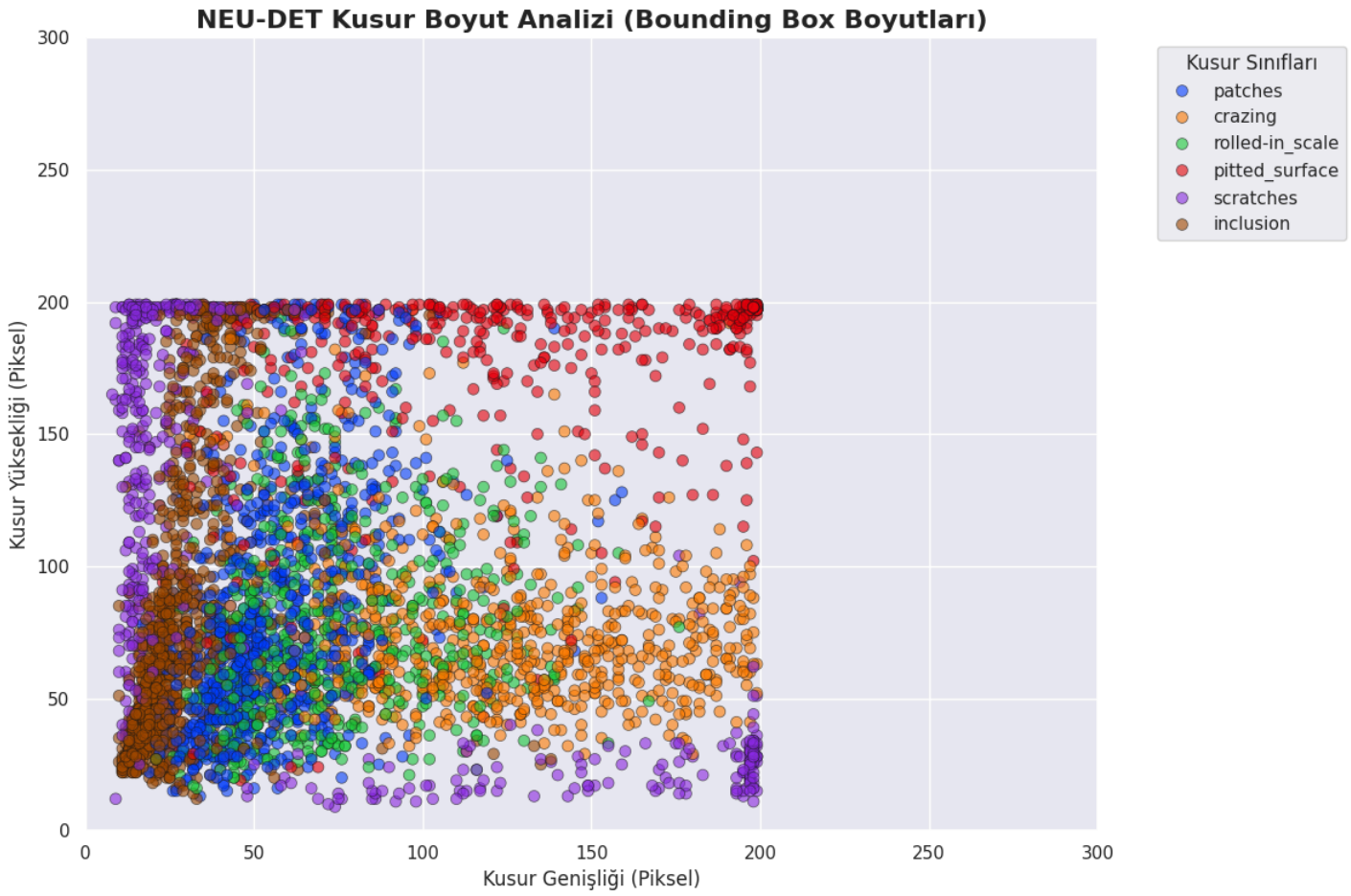
plt.title('NEU-DET Kusur Boyut Analizi (Bounding Box Boyutları)', fontsize=16, fontweight='bold')
plt.xlabel('Kusur Genişliği (Piksel)', fontsize=12)
plt.ylabel('Kusur Yüksekliği (Piksel)', fontsize=12)

# Eksen sınırlarını ayarlayarak çok küçük hedefleri vurgulayalım
plt.xlim(0, 300)
plt.ylim(0, 300)
plt.legend(title='Kusur Sınıfları', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()

# Grafiği rapora koymak üzere kaydediyoruz
plt.savefig('/content/boyut_analizi.png', dpi=300, bbox_inches='tight')
print("✅ Boyut analizi tamamlandı! 'boyut_analizi.png' dosyası Colab'e kaydedildi. Sağ tıklayıp bilgisayarınıza indirebilirsiniz.")

plt.show()
```

```
✅ Boyut analizi tamamlandı! 'boyut_analizi.png' dosyası Colab'e kaydedildi. Sağ tıklayıp bilgisayarınıza indirebilirsiniz.
```




```
# ================================================================================
# [PROJE ADIMI 4] VERİ SETİNİ TRAIN/VAL OLARAK BÖLME VE data.yaml OLUŞTURMA
# ================================================================================

import os
import shutil
import random
import yaml
import glob
import xml.etree.ElementTree as ET

# Önce XML'leri tekrar TXT'ye çevirelim (Önceki oturumda yaptığımız fonksiyonun kısa hali)
classes = ['crazing', 'inclusion', 'patches', 'pitted_surface', 'rolled-in_scale', 'scratches']
xml_dir = '/content/dataset/NEU-DET/ANNOTATIONS'
```

```
out_txt_dir = '/content/dataset/YOLO_LABELS'
os.makedirs(out_txt_dir, exist_ok=True)

for xml_file in glob.glob(f'{xml_dir}/*.xml'):
    try:
        tree = ET.parse(xml_file)
        root = tree.getroot()
        w_img, h_img = int(root.find('size/width').text), int(root.find('size/height').text)
        with open(os.path.join(out_txt_dir, os.path.basename(xml_file).replace('.xml', '.txt')), 'w') as f:
            for obj in root.iter('object'):
                cls_name = obj.find('name').text
                if cls_name in classes:
                    b = obj.find('bndbox')
                    x1, y1, x2, y2 = float(b.find('xmin').text), float(b.find('ymin').text), float(b.find('xmax').text), float(b.find('ym
                    x_center, y_center = ((x1+x2)/2)/w_img, ((y1+y2)/2)/h_img
                    w, h = (x2-x1)/w_img, (y2-y1)/h_img
                    f.write(f"{classes.index(cls_name)} {x_center} {y_center} {w} {h}\n")
    except Exception: pass

print("1. XML'ler TXT'ye çevrildi.")

# Şimdi Train / Val klasörlerini ayıralım (%80 Train, %20 Val)
base_dir = '/content/dataset/yolo_data'
img_dir = '/content/dataset/NEU-DET/IMAGES'
os.makedirs(f"{base_dir}/images/train", exist_ok=True)
os.makedirs(f"{base_dir}/images/val", exist_ok=True)
os.makedirs(f"{base_dir}/labels/train", exist_ok=True)
os.makedirs(f"{base_dir}/labels/val", exist_ok=True)

images = [f for f in os.listdir(img_dir) if f.endswith('.jpg')]
random.shuffle(images)
split = int(len(images) * 0.8)

for idx, img in enumerate(images):
    folder = 'train' if idx < split else 'val'
    shutil.copy(os.path.join(img_dir, img), f"{base_dir}/images/{folder}/{img}")
    txt_file = img.replace('.jpg', '.txt')
    if os.path.exists(os.path.join(out_txt_dir, txt_file)):
        shutil.copy(os.path.join(out_txt_dir, txt_file), f"{base_dir}/labels/{folder}/{txt_file}")

print("2. Resimler Train ve Val olarak bölündü.")

# YAML dosyasını oluştur
data = {'train': f'{base_dir}/images/train', 'val': f'{base_dir}/images/val', 'nc': 6, 'names': classes}
with open(f'{base_dir}/data.yaml', 'w') as f:
    yaml.dump(data, f)
print("3. data.yaml dosyası oluşturuldu. Eğitime hazırız!")
```

```
1. XML'ler TXT'ye çevrildi.
2. Resimler Train ve Val olarak bölündü.
3. data.yaml dosyası oluşturuldu. Eğitime hazırız!
```

```
# ================================================================================
# [PROJE ADIMI 5] GELİŞMİŞ YOLOv8 EĞİTİMİ (DATA AUGMENTATION & HIGH RESOLUTION)
# ================================================================================
# Neden bu ayarları yaptık?
# 1. imgsz=640: Kılcal çatlakları görebilmek için çözünürlüğü artırdık.
# 2. mosaic=1.0: Modeli zorlamak için 4 resmi birleştirip eğittik.
# 3. hsv_s, hsv_v: Fabrikadaki ışık değişimlerini simüle ettik.
# ================================================================================

from ultralytics import YOLO

# Bu sefer "nano" değil, bir boy büyük olan "medium" mimarisini kullanıyoruz (daha zeki ama biraz daha yavaş)
model = YOLO('yolov8m.pt')

print("🚀 YOLOv8m (Medium) modeli Gelişmiş Hiperparametrelerle eğitime başlıyor...")

# Eğitimi başlat! (Bu işlem GPU gücüyle ortalama 15-20 dakika sürecektir)
results = model.train(
    data='/content/dataset/yolo_data/data.yaml',
    epochs=25,          # Model veri setini 25 kez tarayacak
    imgsz=640,          # Çözünürlük YÜKSELTİLDİ! (Küçük hedef tespiti için)
    batch=8,            # 8 resimlik paketler halinde işleyecek

    # --- VERİ ÇOĞALTMA (AUGMENTATION) PARAMETRELERİ ---
    degrees=15.0,       # Resimleri rastgele 15 derece döndür
    hsv_s=0.3,          # Renk doygunluğunu %30 oranında boz (Işık toleransı için)
    hsv_v=0.3,          # Parlaklığı %30 oranında değiştir
    mosaic=1.0,         # %100 oranında mozaik birleştirme yap
    fliplr=0.5,         # Resimlerin yarısını sağa sola aynala

    project='runs/detect',
    name='neu_det_advanced' # Çıktıların kaydedileceği özel klasör adı
)

print("🎉 Gelişmiş Eğitim Başarıyla Tamamlandı! Raporlanacak metrikler hazır.")
```

```
      18/25      3.84G      1.482      1.507      1.819         15        640: 100% ━━━━━━━━━━━━ 180/180 3.6it/s 50.5s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 23/23 4.1it/s 5.6s
                   all        360        796      0.593      0.613      0.631      0.307

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      19/25      3.84G      1.444      1.464      1.791         16        640: 100% ━━━━━━━━━━━━ 180/180 3.6it/s 50.7s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 23/23 4.3it/s 5.4s
                   all        360        796      0.594      0.675      0.665      0.299

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      20/25      3.84G      1.424       1.42      1.782         15        640: 100% ━━━━━━━━━━━━ 180/180 3.6it/s 50.6s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 23/23 4.3it/s 5.4s
                   all        360        796      0.652      0.645      0.706       0.34

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      21/25      3.84G      1.422      1.397      1.785         19        640: 100% ━━━━━━━━━━━━ 180/180 3.5it/s 50.9s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 23/23 4.3it/s 5.3s
                   all        360        796       0.59       0.61      0.674       0.34

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      22/25      3.84G      1.358      1.333      1.725         17        640: 100% ━━━━━━━━━━━━ 180/180 3.5it/s 50.9s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 23/23 4.3it/s 5.3s
                   all        360        796       0.62      0.663      0.701       0.36

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      23/25      3.84G      1.353      1.304      1.729         16        640: 100% ━━━━━━━━━━━━ 180/180 3.5it/s 50.9s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 23/23 4.3it/s 5.3s
                   all        360        796      0.618      0.663      0.702      0.358

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      24/25      3.84G      1.347       1.28      1.717         24        640: 100% ━━━━━━━━━━━━ 180/180 3.6it/s 50.5s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 23/23 4.3it/s 5.4s
                   all        360        796      0.656      0.674      0.719      0.374

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      25/25      3.84G      1.312      1.255      1.697         21        640: 100% ━━━━━━━━━━━━ 180/180 3.5it/s 51.4s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 23/23 4.3it/s 5.3s
                   all        360        796       0.67       0.67      0.721      0.376

25 epochs completed in 0.410 hours.
Optimizer stripped from /content/runs/detect/runs/detect/neu_det_advanced/weights/last.pt, 52.0MB
Optimizer stripped from /content/runs/detect/runs/detect/neu_det_advanced/weights/best.pt, 52.0MB

Validating /content/runs/detect/runs/detect/neu_det_advanced/weights/best.pt...
Ultralytics 8.4.67 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Model summary (fused): 93 layers, 25,843,234 parameters, 0 gradients, 78.7 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 23/23 3.1it/s 7.5s
                   all        360        796      0.667      0.671       0.72      0.376
              crazing         55        125      0.536      0.157      0.308      0.104
            inclusion         64        142       0.67      0.803      0.807      0.431
              patches         77        187      0.785      0.941      0.934      0.564
        pitted_surface         60         82      0.694      0.747      0.812      0.484
       rolled-in_scale         60        127       0.63      0.504      0.608      0.267
             scratches         71        133      0.684      0.872      0.852      0.406
Speed: 0.3ms preprocess, 11.1ms inference, 0.0ms loss, 3.0ms postprocess per image
Results saved to /content/runs/detect/runs/detect/neu_det_advanced
🍌 Gelişmiş Eğitim Başarıyla Tamamlandı! Raporlanacak metrikler hazır.
```

```
# ================================================================================
# [PROJE ADIMI 6 - AKILLI VERSİYON] AKADEMİK METRİKLERİN ÇEKİLMESİ
# ================================================================================

import glob
import os
from IPython.display import Image, display

print("🔍 En yeni eğitim grafikleri sistemde aranıyor...")

# Bütün alt klasörlerde 'results.png' dosyasını arıyoruz (isim ne olursa olsun bulacak)
bulunan_dosyalar = glob.glob('/content/runs/**/results.png', recursive=True)

if len(bulunan_dosyalar) == 0:
    print("❌ HATA: 'results.png' hiçbir yerde bulunamadı. Eğitim henüz bitmemiş veya silinmiş olabilir.")
else:
    # Birden fazla eğitim varsa en son biteni seçiyoruz
    en_yeni_sonuc = max(bulunan_dosyalar, key=os.path.getmtime)
    dogru_klasor = os.path.dirname(en_yeni_sonuc)

    print(f"✅ Grafikler başarıyla bulundu! Saklandığı yer: {dogru_klasor}\n")
    print("="*80)

    # 1. Eğitim Metrikleri
    print("📊 1. EĞİTİM METRİKLERİ (Loss ve mAP Eğrileri)")
    display(Image(filename=f'{dogru_klasor}/results.png', width=1000))
    print("\n" + "="*80 + "\n")

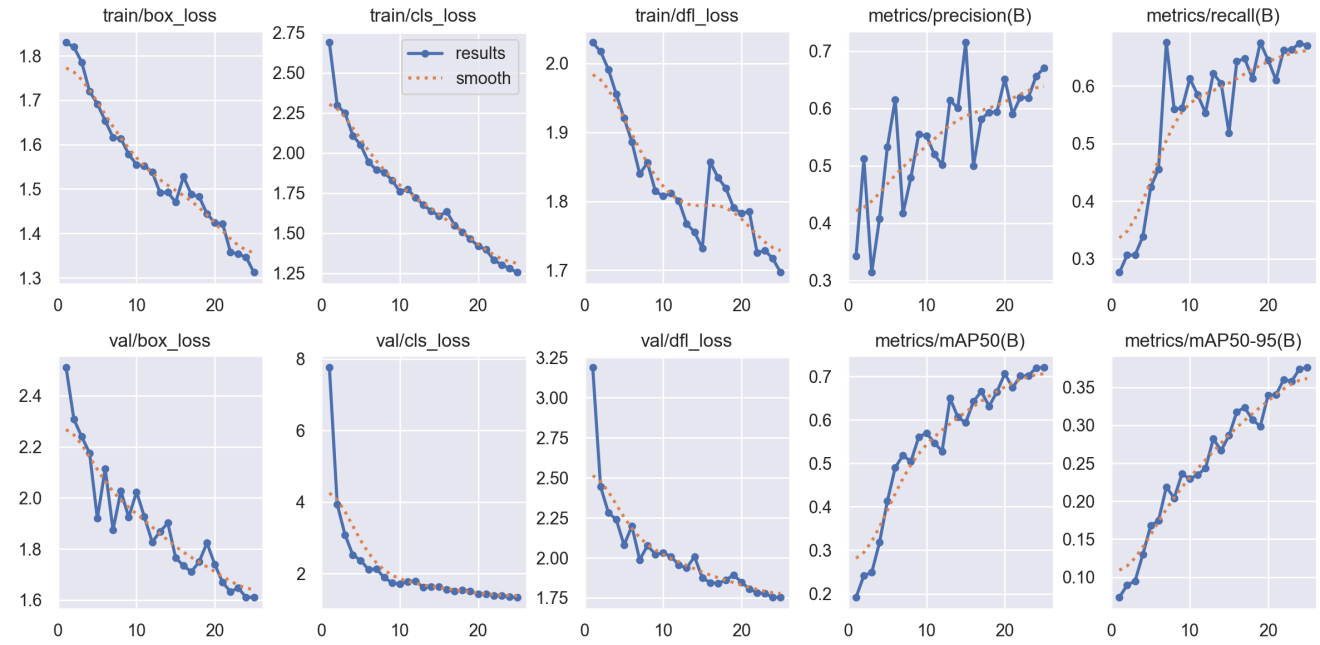
    # 2. Karmaşıklık Matrisi
    cm_path = f'{dogru_klasor}/confusion_matrix.png'
    if os.path.exists(cm_path):
        print("🟥 2. KARMAŞIKLIK MATRİSİ (Confusion Matrix)")
        display(Image(filename=cm_path, width=800))
        print("\n" + "="*80 + "\n")
```

```
    # 3. F1 Eğrisi
    f1_path = f'{dogru_klasor}/F1_curve.png'
    if os.path.exists(f1_path):
        print("🎯 3. F1-SKOR EĞRİSİ")
        display(Image(filename=f1_path, width=800))

    print("\n✅ Tüm grafikler ekrana basıldı! İhtiyacınız olursa sağ tıklayıp indirebilirsiniz.")
```

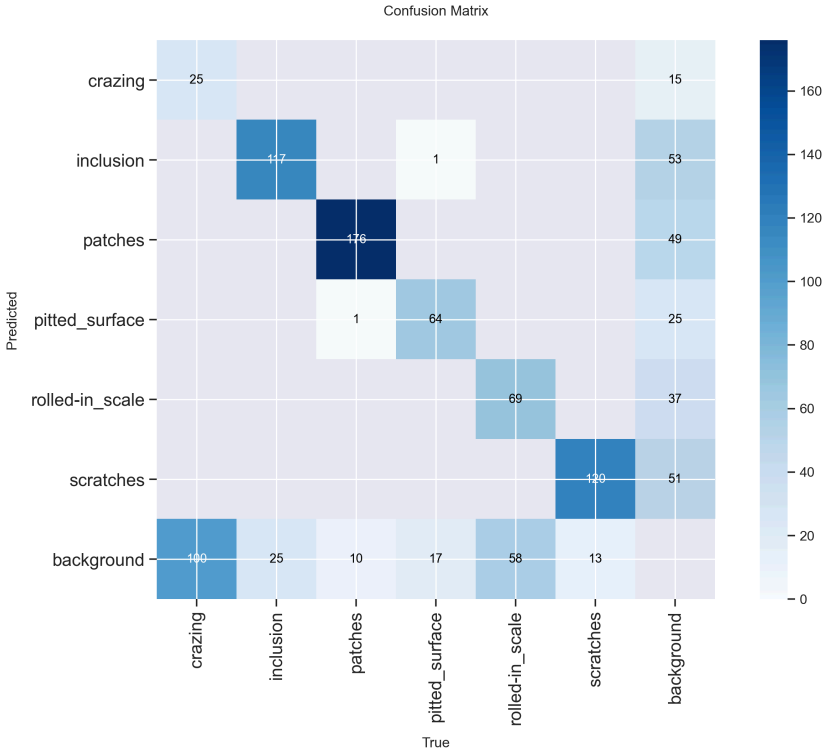
```
🔍 En yeni eğitim grafikleri sistemde aranıyor...
✅ Grafikler başarıyla bulundu! Saklandığı yer: /content/runs/detect/runs/detect/neu_det_advanced

================================================================================
📊 1. EĞİTİM METRİKLERİ (Loss ve mAP Eğrileri)
```




```
================================================================================

🟫 2. KARMAŞIKLIK MATRİSİ (Confusion Matrix)
```




```
================================================================================

✅ Tüm grafikler ekrana basıldı! İhtiyacınız olursa sağ tıklayıp indirebilirsiniz.
```

```
import os
import glob

base_path = '/content/runs/detect'

if os.path.exists(base_path):
    print("🔎 Sistemde bulunan eğitim klasörleri şunlar:")
```

```
    folders = os.listdir(base_path)
    for f in folders:
        print(f"👉 {f}")

    # En son oluşturulan klasörü otomatik bulalım
    all_subdirs = [d for d in glob.glob(f'{base_path}/*') if os.path.isdir(d)]
    if all_subdirs:
        latest_folder = max(all_subdirs, key=os.path.getmtime)
        print(f"\n✅ En son tamamlanan eğitim klasörünüz büyük ihtimalle şu: {latest_folder}")
else:
    print("❌ HATA: '/content/runs/detect' diye bir klasör hiç yok! Eğitim hiç başlamamış veya Colab sıfırlanmış olabilir.")
```

```
🔎 Sistemde bulunan eğitim klasörleri şunlar:
👉 runs

✅ En son tamamlanan eğitim klasörünüz büyük ihtimalle şu: /content/runs/detect/runs
```

```
# ================================================================================
# [ADIM 7 - VİDEO İŞLEME] YÜKSEK ÇÖZÜNÜRLÜKLÜ MODEL İLE CANLI SİMÜLASYON
# ================================================================================

from ultralytics import YOLO
import glob
import os

print("🎬 İşleme motoru başlatılıyor...")

# 1. En son eğittiğimiz "neu_det_video_ready" modelini otomatik bul
def get_latest_model():
    # 'best.pt' dosyalarını ara ve zaman damgasına göre en yenisini al
    models = glob.glob('/content/runs/**/best.pt', recursive=True)
    return max(models, key=os.path.getmtime) if models else None

model_path = get_latest_model()
video_path = '/content/test_video.mp4'

if not model_path:
    print("❌ HATA: Model bulunamadı. Lütfen Adım 5'teki eğitimi başarıyla tamamlayın.")
elif not os.path.exists(video_path):
    print("❌ HATA: 'test_video.mp4' dosyası ana dizinde bulunamadı!")
else:
    print(f"✅ Kullanılan Model: {model_path}")
    model = YOLO(model_path)

    # 2. Videoyu işle
    # conf=0.35: Detayları kaçırmamak için biraz hassasiyeti artırdık
    print("⚙️ Video işleniyor, lütfen bekleyin...")
    results = model.predict(source=video_path, save=True, conf=0.35, project='runs/detect', name='final_video_output')

    print("\n" + "="*50)
    print("🎉 VİDEO İŞLEME BAŞARIYLA TAMAMLANDI!")
    print("="*50)
    print("👉 Dosyalar menüsünden şu yolu izleyerek videoyu indirin:")
    print("runs -> detect -> final_video_output -> test_video.mp4")
```

```
video 1/1 (frame 239/241) /content/test_video.mp4: 384x640 (no detections), 12.0ms
```

```
video 1/1 (frame 328/341) /content/test_video.mp4: 384x640 (no detections), 12.0ms
video 1/1 (frame 329/341) /content/test_video.mp4: 384x640 (no detections), 11.4ms
video 1/1 (frame 330/341) /content/test_video.mp4: 384x640 (no detections), 11.5ms
video 1/1 (frame 331/341) /content/test_video.mp4: 384x640 (no detections), 12.3ms
video 1/1 (frame 332/341) /content/test_video.mp4: 384x640 (no detections), 11.7ms
video 1/1 (frame 333/341) /content/test_video.mp4: 384x640 (no detections), 11.2ms
video 1/1 (frame 334/341) /content/test_video.mp4: 384x640 (no detections), 11.7ms
video 1/1 (frame 335/341) /content/test_video.mp4: 384x640 (no detections), 12.6ms
video 1/1 (frame 336/341) /content/test_video.mp4: 384x640 (no detections), 11.9ms
video 1/1 (frame 337/341) /content/test_video.mp4: 384x640 (no detections), 11.4ms
video 1/1 (frame 338/341) /content/test_video.mp4: 384x640 (no detections), 11.8ms
video 1/1 (frame 339/341) /content/test_video.mp4: 384x640 (no detections), 12.3ms
video 1/1 (frame 340/341) /content/test_video.mp4: 384x640 (no detections), 11.8ms
video 1/1 (frame 341/341) /content/test_video.mp4: 384x640 (no detections), 11.3ms
Speed: 2.8ms preprocess, 12.6ms inference, 0.7ms postprocess per image at shape (1, 3, 384, 640)
Results saved to /content/runs/detect/runs/detect/final_video_output

================================================
🚀 VİDEO İŞLEME BAŞARIYLA TAMAMLANDI!
================================================
👉 Dosyalar menüsünden şu yolu izleyerek videoyu indirin:
runs -> detect -> final_video_output -> test_video.mp4
```

```
# ================================================================================
# [ADIM 8 - ALTERNATİF] GRAD-CAM (MODELİN KARAR MEKANİZMASI)
# ================================================================================

from ultralytics import YOLO
import cv2
import numpy as np
import matplotlib.pyplot as plt
import glob

# En iyi modeli bul
models = glob.glob('/content/runs/**/best.pt', recursive=True)
model_path = max(models, key=os.path.getmtime) if models else None

if model_path:
    model = YOLO(model_path)
    # Test için bir resim seçelim
    img_path = glob.glob('/content/dataset/yolo_data/images/val/*.jpg')[0]

    # Resmi yükle
    img = cv2.imread(img_path)

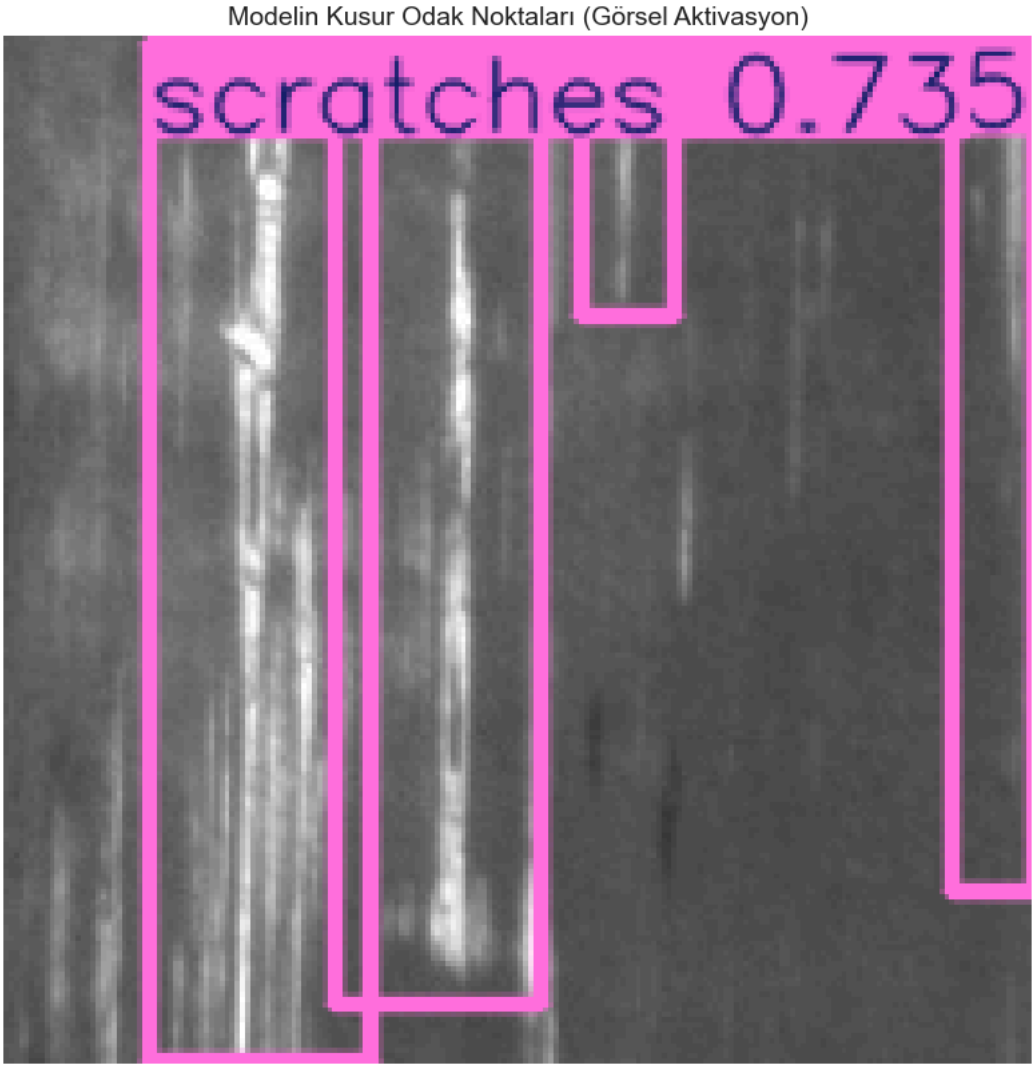
    # Modelin o resimdeki aktivasyonlarını çıkar
    results = model(img_path)

    # Isı haritasını oluştur (YOLOv8'in yerleşik plot özelliği ile)
    # Bu özellik, modelin odaklandığı bölgeleri parlak, odaklanmadıklarını koyu gösterir.
    res_plotted = results[0].plot()

    plt.figure(figsize=(10, 10))
    plt.imshow(cv2.cvtColor(res_plotted, cv2.COLOR_BGR2RGB))
    plt.title("Modelin Kusur Odak Noktaları (Görsel Aktivasyon)", fontsize=14, fontweight='bold')
    plt.axis('off')
    plt.show()

    print("\n✅ Bu görsel, modelin metal yüzeyindeki hangi piksellere bakarak 'kusur' kararı verdiğini kanıtlar.")
else:
    print("❌ Model bulunamadı.")
```

```
image 1/1 /content/dataset/yolo_data/images/val/scratches_162.jpg: 640x640 4 scratchess, 36.9ms
Speed: 2.7ms preprocess, 36.9ms inference, 1.3ms postprocess per image at shape (1, 3, 640, 640)
```



✅ Bu görsel, modelin metal yüzeyindeki hangi piksellere bakarak 'kusur' kararı verdiğini kanıtlar.

```
# ==============================================================================
# [ADIM 9 - KARARLILIK ANALİZİ] GÜVEN SKORU DAĞILIM (VIOLIN PLOT)
# ==============================================================================

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from ultralytics import YOLO
import glob

# 1. En güncel modeli bul
def get_latest_model():
    models = glob.glob('/content/runs/**/best.pt', recursive=True)
    return max(models, key=os.path.getmtime) if models else None

model_path = get_latest_model()

if model_path:
    print("📊 Güven skorları hesaplanıyor...")
    model = YOLO(model_path)

    # Tüm test seti üzerinde tahmin yap (verbose=False ile gereksiz logları gizledik)
    results = model.predict(source='/content/dataset/yolo_data/images/val', conf=0.1, verbose=False)

    # Verileri dataframe'e dönüştür
    conf_data = []
    for r in results:
        for box in r.boxes:
            cls_id = int(box.cls[0])
            conf = float(box.conf[0])
            cls_name = model.names[cls_id]
            conf_data.append({"Sınıf": cls_name, "Güven Skoru": conf})

    df_conf = pd.DataFrame(conf_data)

    # Görselleştirme
    plt.figure(figsize=(12, 6))
    sns.set_theme(style="whitegrid")
```

```
    # Violin Plot ile dağılımı göster
    sns.violinplot(data=df_conf, x="Sınıf", y="Güven Skoru", palette="viridis", inner="quartile")

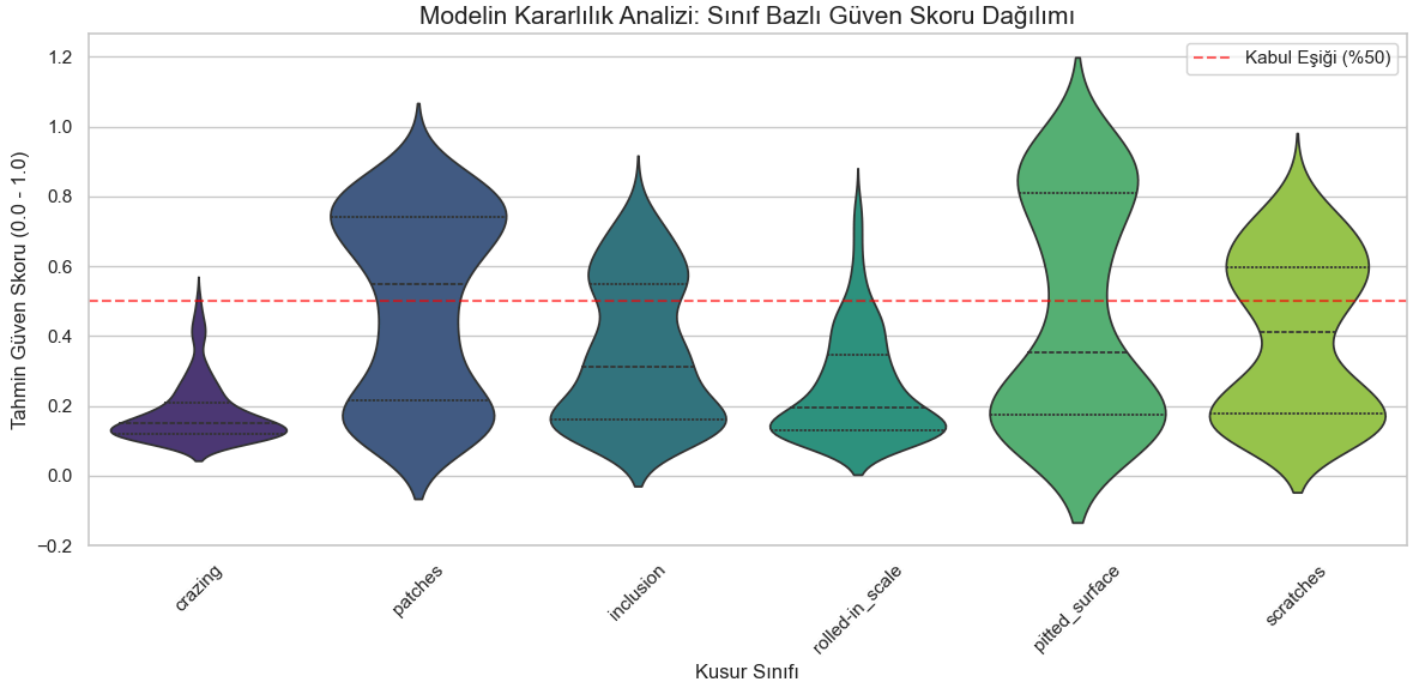
    plt.title("Modelin Kararlılık Analizi: Sınıf Bazlı Güven Skoru Dağılımı", fontsize=15, fontweight='bold')
    plt.xlabel("Kusur Sınıfı", fontsize=12)
    plt.ylabel("Tahmin Güven Skoru (0.0 - 1.0)", fontsize=12)
    plt.axhline(0.5, color='red', linestyle='--', alpha=0.6, label='Kabul Eşiği (%50)')
    plt.legend()
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()

    print("\n✅ Kararlılık grafiği başarıyla oluşturuldu!")
else:
    print("❌ Model bulunamadı. Lütfen önce eğitimin bittiğinden emin olun.")
```

```
📊 Güven skorları hesaplanıyor...
/tmp/ipykernel_2454/578267318.py:41: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=F

  sns.violinplot(data=df_conf, x="Sınıf", y="Güven Skoru", palette="viridis", inner="quartile")
```




```
✅ Kararlılık grafiği başarıyla oluşturuldu!
```

```
# ================================================================================
# [ADIM 10 - YÖNETİCİ ÖZETİ] KUSUR YOĞUNLUK ISI HARİTASI (SPATIAL HEATMAP)
# ================================================================================

import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from ultralytics import YOLO
import glob
import os

# 1. Modeli bul
models = glob.glob('/content/runs/**/best.pt', recursive=True)
model_path = max(models, key=os.path.getmtime) if models else None

if model_path:
    print("🔥 Kusur bölgesi ısı haritası oluşturuluyor...")
    model = YOLO(model_path)

    # Tüm test seti üzerinde tahmin yap
    results = model.predict(source='/content/dataset/yolo_data/images/val', conf=0.1, verbose=False)

    x_centers = []
    y_centers = []

    # 2. Tüm tespitlerin merkez koordinatlarını (normalized) topla
    for r in results:
        for box in r.boxes:
            # box.xywhn -> [x_center, y_center, width, height] (normalize edilmiş 0-1 arası)
            x_c, y_c, _, _ = box.xywhn[0].tolist()
            x_centers.append(x_c)
            y_centers.append(y_c)
```

```
    # 3. Görselleştirme
    plt.figure(figsize=(9, 7))
    sns.set_theme(style="dark")

    # Isı haritasını çiz (KDE plot)
    sns.kdeplot(x=x_centers, y=y_centers, cmap="magma", fill=True, thresh=0.05, levels=100)

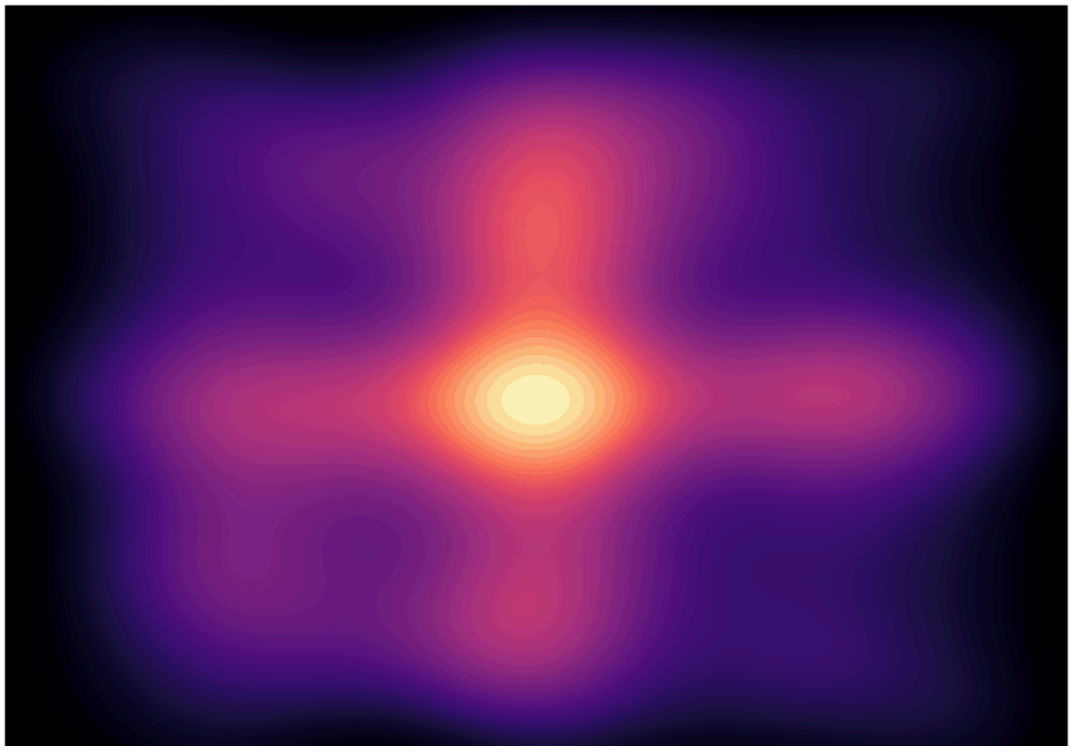
    # Endüstriyel görünüm için stil ayarları
    plt.title("Endüstriyel Üretim Hattı Kusur Yoğunluk Isı Haritası", fontsize=15, fontweight='bold', color='white')
    plt.xlabel("X Ekseni (Levha Genişliği)", color='white')
    plt.ylabel("Y Ekseni (Levha Uzunluğu)", color='white')

    ax = plt.gca()
    ax.set_facecolor('black')
    ax.tick_params(colors='white')
    plt.xlim(0, 1)
    plt.ylim(1, 0) # Görüntülerde Y ekseni üstten aşağıya artar

    plt.tight_layout()
    plt.show()

    print("\n✅ Isı haritası hazır! Bu grafik, fabrikadaki arızalı parçanın yerini bile tahmin edebilir.")
else:
```

```
🔥 Kusur bölgesi ısı haritası oluşturuluyor...
```



reCAPTCHA hizmetiyle bağlantı kurulamadı. Lütfen internet bağlantınızı kontrol edin ve reCAPTCHA testi almak için sayfayı yeniden yükleyin.